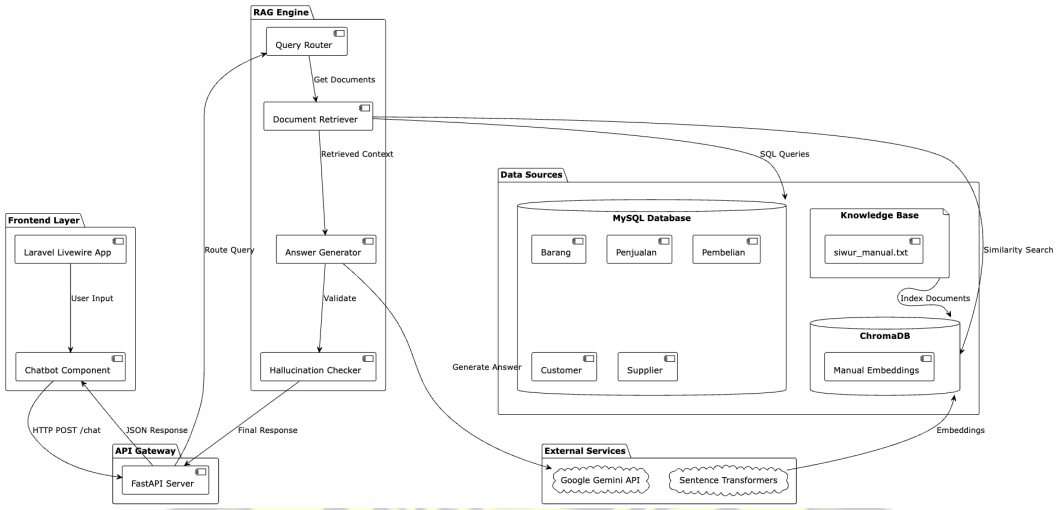
Kode	Kebutuhan
[SRS-AI-015]	Sistem harus melakukan *auto-rebuild index* jika file manual berubah.
[SRS-AI-016]	Sistem harus memiliki *fallback keyword* search jika *vector search* gagal.
[SRS-AI-017]	Sistem harus menghasilkan jawaban dalam Bahasa Indonesia.

4.2 Desain Sistem

Setelah semua kebutuhan didefinisikan pada tahap perencanaan kebutuhan sistem, penelitian memasuki tahap Desain Sistem. Tahap ini merupakan implementasi dari fase kedua metodologi RAD, di mana semua kebutuhan abstrak tersebut diterjemahkan menjadi rancangan teknis dan cetak biru *(blueprint)*. Desain ini menjadi panduan utama untuk tahap pengembangan. Fase desain ini berfokus pada empat area utama, yaitu arsitektur sistem secara keseluruhan, alur proses, desain komponen dan modul, serta desain basis pengetahuan.

4.2.1 Arsitektur Sistem

Arsitektur sistem dirancang menggunakan pendekatan *microservice*. Arsitektur ini, seperti yang divisualisasikan pada Gambar 5, membagi sistem menjadi lima komponen utama yang saling terhubung melalui antarmuka API.



Gambar 5. Diagram komponen arsitektur Siwur

Komponen-komponen utama dalam arsitektur ini adalah:

a. *Frontend Layer*

Frontend Layer pada gambar 5 merepresentasikan aplikasi web Siwur yang dibangun dengan TALL Stack, di mana Laravel Livewire mengelola *state* percakapan dan mengirim pertanyaan pengguna dari komponen *chatbot* ke *backend* tanpa reload halaman. Alpine.js dipakai untuk interaksi ringan seperti membuka/menutup jendela *chat*, animasi "bot sedang mengetik", dan binding event tombol, sementara Tailwind CSS dan Blade *template* bertanggung jawab penuh atas tampilan responsif dan struktur markup komponen chat. Lapisan ini hanya menangani UI dan meneruskan pertanyaan ke *API Gateway* melalui *HTTP POST*, lalu menampilkan kembali jawaban yang diterima dalam bentuk bubble pesan di jendela chat.

b. *API Gateway*

API Gateway digambarkan sebagai server FastAPI yang menjadi pintu masuk semua request dari *frontend* ke layanan AI, dengan *endpoint* utama /chat untuk memproses pertanyaan *chatbot* dan mengembalikan jawaban dalam format JSON berisi teks jawaban, datasource, jumlah dokumen, dan waktu proses. *Endpoint* tambahan seperti /mba digunakan untuk menjalankan *Market Basket Analysis,* sementara *endpoint health check* dan *rebuild-index* mendukung *monitoring* serta pemicu *rebuild indeks* ChromaDB ketika data berubah. Komponen ini juga menangani validasi payload (misalnya batas panjang *question* dan keberadaan *tenant id*), penanganan error, dan pemberian kode status HTTP yang konsisten agar antarmuka klien tetap stabil.

c. *RAG Engine*

RAG Engine pada diagram adalah inti pemrosesan AI yang diimplementasikan sebagai Adaptive RAG Pipeline: pertanyaan yang masuk pertama kali dianalisis oleh *Query Router* untuk memutuskan jalur *datasource (database, manual, analytics*, atau *hybrid)* sebelum diteruskan ke modul-modul skill spesifik. *Skills System (DatabaseSkill, ManualSkill, AnalyticsSkill)* melakukan *retrieval* ke masing-masing sumber, kemudian *Document Retriever* menggabungkan dan memberikan dokumen ke *modul grading* yang menyaring hanya konteks paling relevan, sehingga meminimalkan *noise* sebelum generasi jawaban. Setelah Gemini menghasilkan jawaban, *Hallucination Checker* membandingkan jawaban

dengan konteks untuk mendeteksi ketidaksesuaian, dan bila perlu memicu *re-generation* sehingga jawaban akhir yang dikirim kembali ke *API Gateway* tetap konsisten dengan data.

d. *Data Sources*

Data Sources pada gambar mencakup MySQL Database, *Knowledge Base* teks, dan ChromaDB sebagai *vector store* yang saling terhubung untuk menyediakan kombinasi data faktual dan pengetahuan deskriptif. MySQL menyimpan data transaksi *real-time* seperti tabel barang, stok gudang, penjualan, pembelian, customer, dan *supplier* yang diakses lewat *query* yang sudah ditetapkan oleh DatabaseSkill untuk menjawab pertanyaan faktual seperti stok, omzet, atau profit. *Knowledge Base* berupa file siwur_complete_manual.txt berisi panduan lengkap penggunaan SIWUR yang di-embedding menjadi vektor 384 dimensi dan disimpan di ChromaDB, memungkinkan pencarian semantik atas prosedur, fitur, dan aturan sistem saat pengguna menanyakan pertanyaan terkait panduan penggunaan fitur.

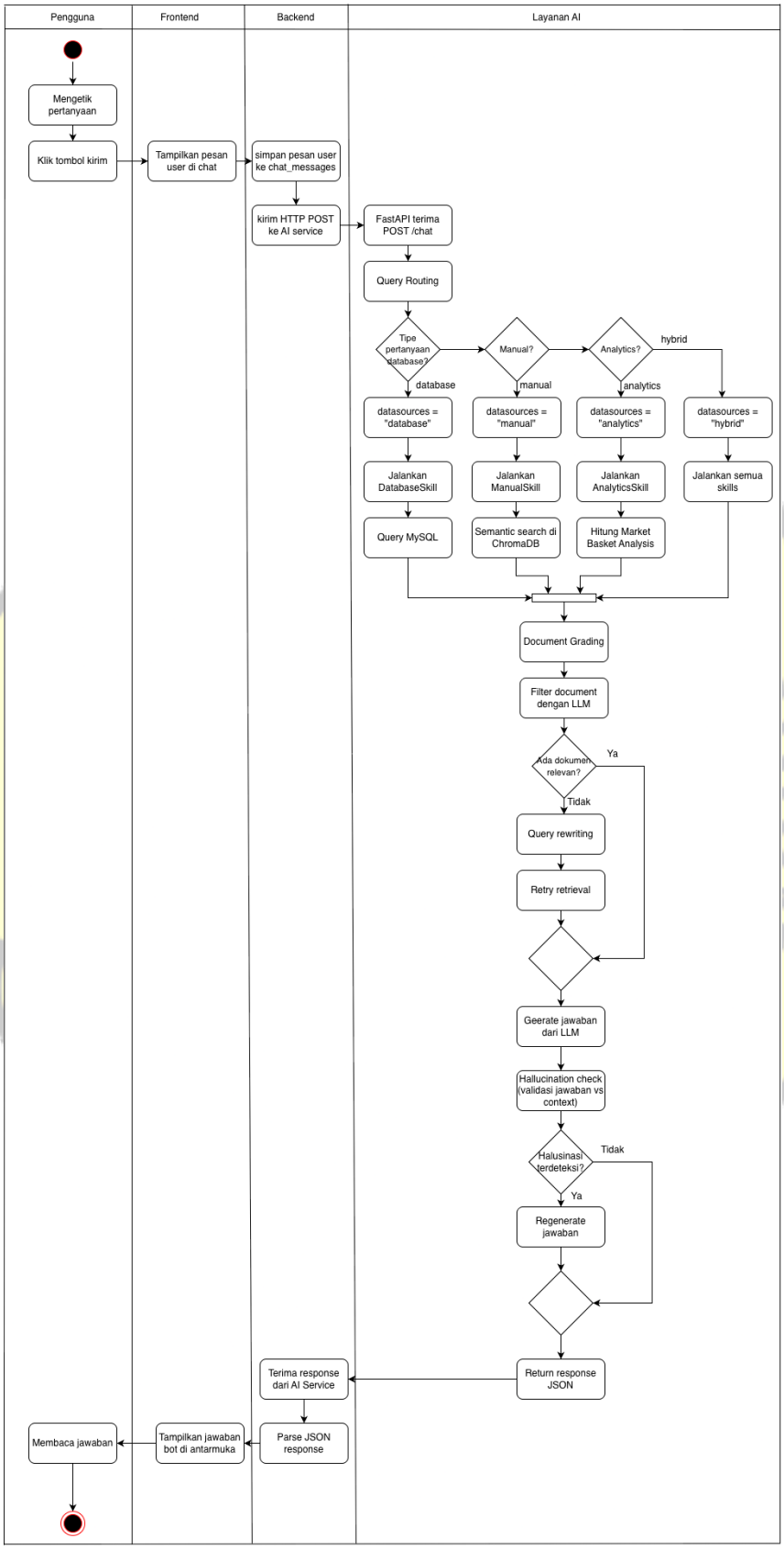
e. *External Services*

External Services pada diagram menunjukkan ketergantungan sistem pada layanan AI eksternal, yaitu Google Gemini 2.0 Flash API sebagai *Large Language Model* yang menerima *prompt* berisi pertanyaan plus konteks terpilih lalu mengembalikan jawaban bahasa alami dalam Bahasa Indonesia. Untuk menghasilkan embedding vektor yang digunakan oleh ChromaDB, sistem memakai library Sentence-Transformers dengan

model all-MiniLM-L6-v2, sehingga teks dari manual maupun dokumen analitik dapat dipetakan ke basis data vektor dan dicari menggunakan *similarity search* secara efisien.

4.2.2 Desain Alur Proses RAG

Setelah arsitektur sistem dirancang, tahap desain selanjutnya adalah merinci alur proses *Retrieval-Augmented Generation (RAG)* seperti yang divisualisasikan secara rinci enggunakan *Activity Diagram* pada Gambar 6.



Gambar 6. *Activity diagram* proses alur RAG Siwur

a. Tahap 1: *User Input* dan Validasi (*Frontend & Backend* Laravel)

- Pengguna mengetikkan pertanyaan pada modal chatbot berbasis Livewire.
- Livewire component mengirimkan permintaan AJAX ke ChatbotController.
- Controller melakukan validasi input dan menentukan toko_id berdasarkan pengguna yang sedang login.
- Pesan dari pengguna disimpan ke tabel chat_messages dengan role = 'user'.

b. Tahap 2: *Proxy Request* ke Layanan AI (FastAPI)

- *Controller* mengirimkan *request* HTTP POST ke *endpoint* AI Service:

```
{
  "question": "Berapa stok Indomie?",
  "tenant_id": "1"
}
```

- FastAPI menerima request tersebut dan mulai memproses *query*.

c. Tahap 3: *Query Routing*

- Pertanyaan diproses oleh modul AdaptiveRAG.route_question().
- Sistem menggunakan Gemini API untuk menganalisis jenis pertanyaan dan menentukan *datasource* yang sesuai (*database, manual, analytics*, dan *hybrid*).

  Contoh:

  Pertanyaan "Berapa stok Indomie?" → diklasifikasikan sebagai *database*.

d. Tahap 4: *Document Retrieval*

- Berdasarkan hasil *routing*, sistem menjalankan *skill retrieval* yang relevan:

  DatabaseSkill

Mendeteksi kata kunci “stok” dan “Indomie”, lalu menjalankan SQL query seperti:

```
SELECT barang.nama_barang, gudang_stock.jumlah,
gudang.nama_gudang, satuan.nama_satuan
FROM barang
JOIN gudang_stock ON barang.id = gudang_stock.barang_id
JOIN gudang ON gudang_stock.gudang_id = gudang.id
JOIN satuan ON barang.satuan_terkecil_id = satuan.id
WHERE barang.toko_id = ?
AND LOWER(barang.nama_barang) LIKE ?
```

ManualSkillI : Melakukan *semantic search* pada koleksi dokumen di ChromaDB.

AnalyticsSkill : Menjalankan *Market Basket Analysis* untuk pertanyaan berbasis analitik.

- Hasil *retrieval* kemudian diformat menjadi objek *document*.

e. Tahap 5: *Document Grading*

- Setiap dokumen diperiksa tingkat relevansinya menggunakan LLM dengan *prompt:*

  “Apakah dokumen ini relevan dengan pertanyaan *{question}*?”

- Dokumen dengan hasil *“yes”* dipertahankan, sedangkan yang *“no”* dibuang.

f. Tahap 6:

Dokumen relevan diubah menjadi sebuah *context string*, misalnya:

```
[Sumber: database]
## Informasi Stok Barang
- Indomie Goreng (IND-001): 500 pcs – Gudang Utama
- Indomie Soto (IND-002): 200 pcs – Gudang Utama
[Sumber: manual]
## Cara Melihat Stok Barang
...
```

g. Tahap 7:

- Sistem membentuk prompt lengkap berisi konteks dan pertanyaan pengguna, lalu mengirimkannya ke Gemini:

```
Anda adalah asisten AI untuk sistem SIWUR.
Konteks:
[...context...]
Pertanyaan: Berapa stok Indomie?
Jawaban:
```

- Gemini menghasilkan jawaban, misalnya:

```
Indomie Goreng: 500 pcs
Indomie Soto: 200 pcs
Total: 700 pcs
```

h. Tahap 8:

- Jawaban diverifikasi kembali:

  "Apakah jawaban ini sesuai dengan konteks?"

- Jika terdeteksi halusinasi dan jumlah *retry* kurang dari batas, maka sistem melakukan *regeneration*.

i. Tahap 9:

- Sistem memeriksa apakah jawaban benar-benar menjawab pertanyaan pengguna.
- Jika tidak, dilakukan *query rewriting* dan *retrieval* ulang.

j. Tahap 10:

- FastAPI mengembalikan response JSON ke Laravel:

```
{
  "answer": "Berdasarkan data sistem, stok Indomie saat ini
adalah...",
  "datasource": "database",
  "documents_count": 2,
```

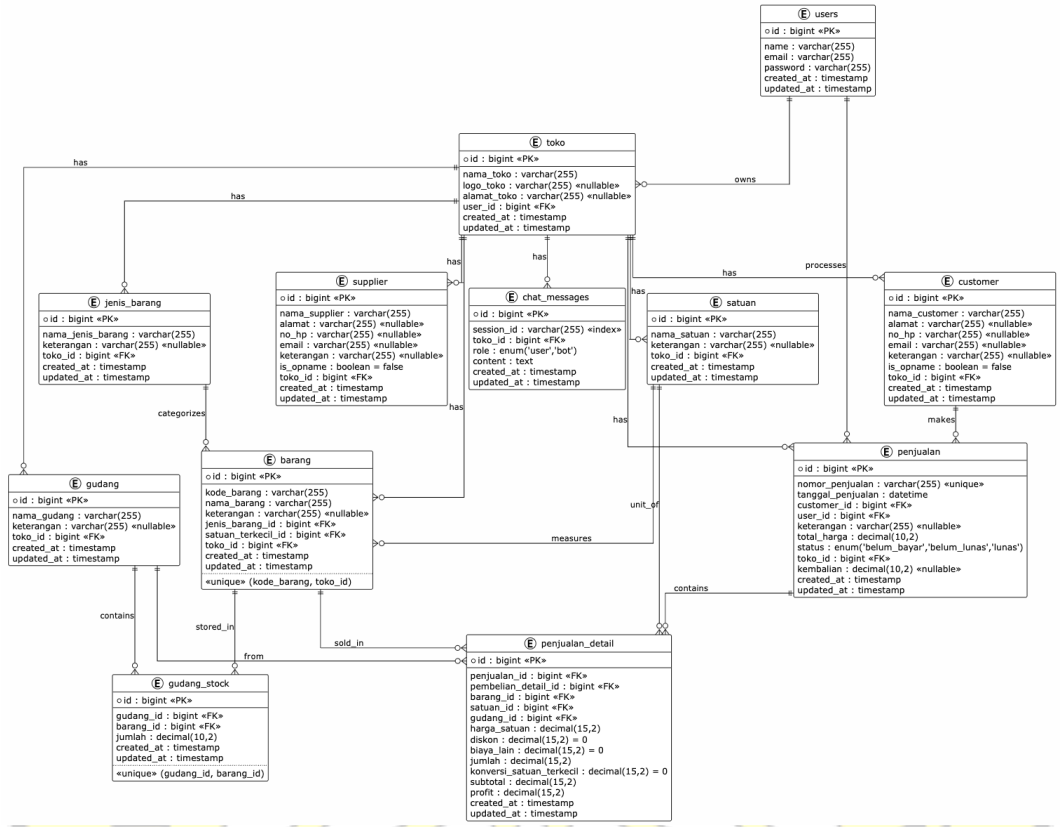
```
  "retry_count": 0
}
```

- *Controller* menyimpan jawaban ke tabel chat_messages dengan role='bot'.
- Livewire menampilkan jawaban pada tampilan chatbot dan otomatis menggulir ke pesan terbaru.

4.2.3 Desain Basis Pengetahuan Asisten Virtual Siwur

Desain basis pengetahuan pada Asisten Virtual SIWUR disusun dengan pendekatan *Adaptive Retrieval-Augmented Generation (Adaptive RAG)* yang memungkinkan sistem memilih dan menggabungkan sumber informasi secara dinamis berdasarkan karakteristik pertanyaan pengguna. Pendekatan ini dirancang agar asisten mampu menyediakan jawaban yang akurat, kontekstual, dan relevan untuk berbagai kebutuhan operasional toko, mulai dari informasi stok barang, panduan penggunaan sistem, hingga analisis performa penjualan. Untuk mencapai hal tersebut, basis pengetahuan Siwur tidak hanya bersifat satu dimensi, tetapi terstruktur ke dalam empat jenis sumber pengetahuan yang saling melengkapi, yaitu pengetahuan operasional dari *database*, pengetahuan prosedural dari dokumen manual, pengetahuan analitis dari modul *analytics*, serta integrasi adaptif melalui mekanisme *hybrid.* Keempat komponen ini membentuk sistem pengetahuan menyeluruh yang mampu mendukung proses pengambilan keputusan, eksekusi operasional, serta penyediaan insight strategis dalam satu kerangka kerja yang kohesif.

a. Basis Pengetahuan Berbasis *Database*



Gambar 8. Tabel Relasi Asisten Virtual Siwur

Basis pengetahuan pertama bersumber dari basis data MySQL, yang berfungsi sebagai repositori informasi operasional sistem Siwur. Data yang dikelola meliputi entitas barang, stok, supplier, customer, transaksi pembelian, transaksi penjualan, serta gudang penyimpanan. Struktur data yang terorganisasi dalam bentuk tabel relasi memungkinkan sistem melakukan penarikan informasi secara langsung melalui eksekusi *query* SQL. Komponen ini menjadi landasan utama untuk menjawab pertanyaan faktual seperti jumlah stok barang, transaksi terbaru, harga barang, atau daftar pemasok yang aktif. Seluruh akses terhadap basis data dilakukan secara langsung melalui *query* SQL yang disusun berdasarkan intent pengguna.

b. Basis Pengetahuan Berbasis Dokumen *Manual Book Siwur*

Sumber pengetahuan kedua berasal dari dokumen siwur_complete_manual.txt, yang berisi seluruh pedoman penggunaan sistem SIWUR, termasuk penjelasan modul, alur kerja operasional, definisi entitas, serta hak akses berdasarkan peran pengguna. Dokumen tersebut diubah menjadi *knowledge chunks* untuk diproses oleh model embedding menggunakan Sentence-Transformers all-MiniLM-L6-v2. Potongan-potongan *embedding* tersebut kemudian disimpan dalam vector store (ChromaDB), sehingga dapat dipanggil melalui mekanisme *semantic search* untuk menjawab pertanyaan prosedural.

c. Basis Pengetahuan Berbasis *Analytics*

Sumber pengetahuan ketiga merupakan pengetahuan analitis yang dihasilkan dari pengolahan data transaksi, dikenal sebagai *analytics knowledge source*. Komponen ini meliputi *Market Basket Analysis*, identifikasi produk terlaris, analisis tren penjualan, profit margin, rekomendasi bundling, hingga peluang *cross-selling*. Sistem menjalankan analisis menggunakan *query* analitik lalu mengubah hasilnya menjadi dokumen kontekstual yang siap digunakan dalam proses RAG. Pengetahuan analitis ini memungkinkan asisten menjawab pertanyaan strategis seperti pola pembelian pelanggan, kombinasi produk yang sering dibeli bersama, performa penjualan dalam 30 hari terakhir, atau peluang peningkatan keuntungan. Dengan demikian*, analytics* menjadi lapisan pengetahuan yang mendukung *insight* prediktif dan rekomendasi bisnis.

d. Basis Pengetahuan *Hybrid* sebagai Integrasi Adaptif

Sumber pengetahuan keempat merupakan gabungan adaptif dari ketiga sumber sebelumnya, yang disebut sebagai *hybrid knowledge source*. Pendekatan hybrid memungkinkan asisten menggabungkan data faktual, panduan prosedural, dan insight analitis untuk menjawab pertanyaan kompleks yang membutuhkan lebih dari satu jenis informasi. Ketika pengguna menanyakan hal yang memerlukan konteks ganda—misalnya strategi penjualan yang tepat untuk produk tertentu berdasarkan stok aktual sekaligus pola pembelian pelanggan—sistem akan mengambil dokumen dari database, manual, dan analytics secara bersamaan. Kerangka Adaptive RAG menentukan secara dinamis sumber mana yang paling relevan berdasarkan intent pertanyaan. Dengan mekanisme ini, Asisten Virtual Siwur mampu memberikan jawaban komprehensif yang menggabungkan kondisi operasional, aturan penggunaan sistem, serta wawasan berbasis data sekaligus.

4.3 Pengembangan Sistem

Pada tahap implementasi sistem, seluruh komponen yang sebelumnya dirancang mulai diwujudkan dalam bentuk proses yang dapat dijalankan secara nyata. Fokus utama pada bagian ini adalah bagaimana sistem mengolah informasi, membangun representasi data yang bermakna, dan melakukan pencarian jawaban secara cerdas berdasarkan konteks. Untuk mencapai hal tersebut, sistem memanfaatkan teknik *embedding* serta mekanisme pencarian semantik yang memungkinkan pencocokan makna secara lebih mendalam dibandingkan pencarian berbasis kata kunci tradisional. Oleh karena itu, sebelum menguraikan alur kerja